

Evidencia de Criptografía Post-Cuántica — Cerulean Ledger

Inventario completo de tests que demuestran que ML-DSA-65 (FIPS 204) está implementado y verificado end-to-end en la plataforma.

Última verificación: 2026-04-23 — todos los tests passing.

Resumen ejecutivo

Categoría	Tests	Estado
ML-DSA-65 unitarios	8	Passing
FIPS 140-3 KAT self-tests	1 (cubre 3 algoritmos)	Passing
Aislamiento entre algoritmos	1	Passing
Property-based (datos aleatorios)	2	Passing
Total tests PQC dedicados	12	Passing
Tests que usan firmas Vec <u>u</u> 8 (PQC-compatible)	~250+	Passing

Capa 1: Criptografía base ML-DSA-65

Archivo: src/identity/signing.rs **Proveedor:** MlDsaSigningProvider (usa pqcrypto-mldsa 0.1.2, implementación de CRYSTALS-Dilithium / FIPS 204)

Tests unitarios (8)

Test	Qué demuestra	Línea
mldsa65_sign_and_verify_roundtrip	Firma ML-DSA-65 se genera y verifica correctamente	signing.rs:336
mldsa65_verify_wrong_data_fails	Datos alterados post-firma son rechazados	signing.rs:344
mldsa65_algorithm_identifier		signing.rs:351

Test	Qué demuestra	Línea
	El proveedor se identifica como SigningAlgorithm::MLDsa65	
mldsa65_signature_is_3309_bytes	La firma tiene exactamente 3,309 bytes (conforme a FIPS 204, security level 3)	signing.rs:357
mldsa65_public_key_is_1952_bytes	La clave pública tiene exactamente 1,952 bytes (conforme a spec)	signing.rs:364
mldsa65_verify_rejects_wrong_signature	Firma incorrecta o truncada es rechazada	signing.rs:370
mldsa65_from_keys_roundtrip	Claves se pueden serializar, persistir y restaurar sin pérdida	signing.rs:379
mldsa65_trait_object_usage	ML-DSA-65 funciona como Box<dyn SigningProvider> — intercambiable con Ed25519 en runtime	signing.rs:390

Cómo ejecutar

```
cargo test mldsa65
```

Capa 2: FIPS 140-3 Known Answer Tests (KAT)

Archivo: src/identity/signing.rs — función run_crypto_self_tests()

Invocación en producción: src/main.rs:762 — se ejecuta al arranque, antes de aceptar requests.

Qué hace run_crypto_self_tests()

Para cada algoritmo: 1. Genera un keypair 2. Firma un test vector conocido (b"FIPS-140-3-KAT-Ed25519", b"FIPS-140-3-KAT-ML-DSA-65") 3. Verifica la firma 4. Corrompe un byte de la firma y verifica que es rechazada 5. Para SHA-256: calcula hash de b"FIPS-140-3-KAT-SHA256" y compara contra valor esperado hardcoded

Si cualquier paso falla, el nodo se niega a arrancar.

Algoritmos cubiertos

Algoritmo	Estándar	Test vector
Ed25519	—	b"FIPS-140-3-KAT-Ed25519"
ML-DSA-65	FIPS 204	b"FIPS-140-3-KAT-ML-DSA-65"
SHA-256	FIPS 180-4	b"FIPS-140-3-KAT-SHA256" → 11ffe3edc...

Test unitario

Test	Línea
crypto_self_tests_pass	signing.rs:271

Cómo ejecutar

```
cargo test crypto_self_tests
```

Capa 3: Aislamiento entre algoritmos

Archivo: src/identity/signing.rs

Test	Qué demuestra	Línea
cross_provider_signatures_incompatible	Una firma Ed25519 (64 bytes) no valida en un proveedor ML-DSA-65, y una firma ML-DSA-65 (3,309 bytes) no valida en un proveedor Ed25519. No hay confusión de algoritmos.	signing.rs:397

Este test verifica que la coexistencia de ambos algoritmos en la misma red no introduce vulnerabilidades de confusión de tipo.

Cómo ejecutar

```
cargo test cross_provider
```

Capa 4: Property-based testing (proptest)

Archivo: src/identity/signing.rs — módulo tests::prop

A diferencia de los tests unitarios que prueban valores fijos, estos tests generan **cientos de casos aleatorios** por ejecución, verificando invariantes criptográficas fundamentales.

Test	Invariante	Rango de datos	Línea
<code>mldsa65_sign_verify_any_data</code>	<code>verify(data, sign(data)) == true</code> para cualquier data	0–1,024 bytes aleatorios	<code>signing.rs:443</code>
<code>mldsa65_verify_rejects_different_data</code>	<code>verify(B, sign(A)) == false</code> para cualquier $A \neq B$	1–512 bytes aleatorios	<code>signing.rs:450</code>

Equivalentes Ed25519 también existen para comparación:

Test	Invariante	Línea
<code>ed25519_sign_verify_any_data</code>	Misma invariante para Ed25519	<code>signing.rs:417</code>
<code>ed25519_verify_rejects_different_data</code>	Misma invariante para Ed25519	<code>signing.rs:424</code>
<code>ed25519_signature_is_deterministic</code>	Misma clave + mismos datos → misma firma	<code>signing.rs:435</code>

Cómo ejecutar

```
cargo test prop
```

Capa 5: Integración end-to-end en la stack

ML-DSA-65 no vive aislado en `signing.rs`. Las firmas de longitud variable (`Vec<u8>`) están integradas en toda la plataforma. Cada struct que lleva firma soporta tanto Ed25519 (64 bytes) como ML-DSA-65 (3,309 bytes).

Campos `Vec<u8>` para firmas PQC-compatible

Struct	Campo	Archivo	Propósito
<code>Block</code>	<code>signature: Vec<u8></code>	<code>src/storage/traits.rs:23</code>	Firma del bloque
<code>Block</code>			

Struct	Campo	Archivo	Propósito
	orderer_signature: Option<Vec<u8>>	src/storage/ traits.rs:29	Firma del orderer
DagBlock	signature: Vec<u8>	src/consensus/ dag.rs:24	Firma en el DAG de consenso
VoteMessage	signature: Vec<u8>	src/consensus/ bft/ types.rs:46	Firma de votos BFT
Endorsement	signature: Vec<u8>	src/ endorsement/ types.rs:44	Firma de endorsement
TransactionProposal	creator_signature: Vec<u8>	src/ transaction/ proposal.rs:14	Firma del creador de la propuesta
AliveMessage	signature: Vec<u8>	src/network/ gossip.rs:37	Firma de mensajes gossip P2P

Auto-detección de algoritmo en transacciones legacy

Archivo: src/models.rs:96-135

La función `Transaction::verify_signature()` auto-detecta el algoritmo por tamaño:

- Clave pública 32 bytes + firma 64 bytes → Ed25519
- Clave pública 1,952 bytes + firma 3,309 bytes → ML-DSA-65
- Cualquier otra combinación → rechazo

Esto permite redes mixtas donde transacciones antiguas (Ed25519) y nuevas (ML-DSA-65) coexisten.

Tests que ejercitan firmas Vec<u8>

Estos tests no prueban PQC directamente, pero operan sobre structs con campos `Vec<u8>` que soportan ambos tamaños de firma:

Suite	Tests	Archivo(s)
Consenso BFT (votos, quorum, rounds)	147	src/consensus/bft/
Network/gossip (alive messages, signatures)	39	src/network/ gossip.rs
Gateway (endorse → order → commit)	30+	src/gateway/mod.rs
Endorsement (validación de firmas)	20+	src/endorsement/
BFT adversario E2E	16	tests/bft_e2e.rs

Suite	Tests	Archivo(s)
Storage (bloques con signature Vec)	50+	src/storage/

Capa 6: Selección de algoritmo en runtime

Archivo: src/main.rs **Variable de entorno:** SIGNING_ALGORITHM

Valor	Algoritmo seleccionado
ed25519 (default)	Ed25519 (64-byte signatures)
ml-dsa-65 o mldsa65	ML-DSA-65 / FIPS 204 (3,309-byte signatures)
Cualquier otro	Fallback a Ed25519 con warning en log

La selección se logea al arranque. Nodos con diferentes algoritmos coexisten en la misma red — la verificación auto-detecta por tamaño.

Cómo ejecutar toda la suite PQC

```
# Tests ML-DSA-65 en crate principal (verify, key validation)
cargo test mldsa65

# Suite completa del crate PQC (72 tests: KAT, self-tests,
  aislamiento, property-based)
cargo test -p pqc_crypto_module
```

Resultado esperado: 74 tests passed (2 main + 72 pqc_crypto_module), 0 failed.

Diseño arquitectónico que habilita PQC

Por qué funciona sin flag day

La decisión de diseño clave fue usar Vec en lugar de [u8; 64] para todos los campos de firma en la plataforma. Esto fue un cambio deliberado respecto al diseño original:

Antes (pre-PQC)	Después (PQC-ready)
signature: [u8; 64]	signature: Vec <u8< u=""></u8<>
Solo Ed25519	Ed25519 (64 bytes) o ML-DSA-65 (3,309 bytes)
Tamaño fijo en serialización	

Antes (pre-PQC)	Después (PQC-ready)
	Serializado como hex string via vec_hex serde helpers

Structs migrados

Los siguientes structs fueron migrados de `[u8; 64]` a `Vec<u8>`:

- `Endorsement.signature`
- `Block.signature` y `Block.orderer_signature`
- `DagBlock.signature`
- `TransactionProposal.creator_signature`
- `AliveMessage.signature` (gossip)

Trait SigningProvider

```
pub trait SigningProvider: Send + Sync {
    fn sign(&self, data: &[u8]) -> Result<Vec<u8>, SigningError>;
    fn verify(&self, data: &[u8], signature: &[u8]) ->
        Result<bool, SigningError>;
    fn public_key(&self) -> Vec<u8>;
    fn algorithm(&self) -> SigningAlgorithm;
}
```

Dos implementaciones: - `SoftwareSigningProvider` — Ed25519 (ed25519-dalek) - `MlDsaSigningProvider` — ML-DSA-65 (pqcrypto-mlDSA, FIPS 204)

Ambas intercambiables via `Box<dyn SigningProvider>`.

Dependencias criptográficas

Crate	Versión	Propósito
ed25519-dalek	—	Ed25519 sign/verify
pqcrypto-mlDSA	0.1.2	ML-DSA-65 (FIPS 204) sign/verify
pqcrypto-traits	0.3	Traits compartidos para PQC
sha2	—	SHA-256 hashing
zeroize	1.7	Zeroización de claves en memoria

Referencia a estándares

Estándar	Qué aplica	Dónde se evidencia
NIST FIPS 204	ML-DSA-65 (CRYSTALS-Dilithium)	MlDsaSigningProvider, tamaños de firma/clave conformes

Estándar	Qué aplica	Dónde se evidencia
NIST FIPS 180-4	SHA-256	KAT self-test con hash esperado hardcoded
FIPS 140-3	Self-tests al arranque	run_crypto_self_tests() en main.rs
CNSS Policy 15	Exigencia de quantum-safe para 2030	ML-DSA-65 implementado y disponible

Documento generado el 2026-04-23. Para reproducir, ejecutar cargo test mldsa65 crypto_self_tests cross_provider desde la raíz del repositorio.